

## Codes

```
GNU nano 8.7.1 memory.c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int i;

    int *a = (int *)malloc(3 * sizeof(int));
    for (i = 0; i < 3; i++)
        a[i] = (i + 1) * 10;

    printf("malloc: ");
    for (i = 0; i < 3; i++)
        printf("%d ", a[i]);
    printf("\n");

    int *b = (int *)calloc(3, sizeof(int));
    printf("calloc: ");
    for (i = 0; i < 3; i++)
        printf("%d ", b[i]);
    printf("\n");

    a = (int *)realloc(a, 5 * sizeof(int));
    a[3] = 40;
    a[4] = 50;

    printf("realloc: ");
    for (i = 0; i < 5; i++)
        printf("%d ", a[i]);
    printf("\n");

    free(a);
    free(b);

    printf("Memory freed successfully.\n");

    return 0;
}
```

## Output:

```
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Practical-08$ ./memory
malloc: 10 20 30
calloc: 0 0 0
realloc: 10 20 30 40 50
Memory freed successfully.
Memory freed successfully.
==1311==
==1311== HEAP SUMMARY:
==1311==    in use at exit: 0 bytes in 0 blocks
==1311==   total heap usage: 4 allocs, 4 frees, 1,068 bytes allocated
==1311==
==1311== All heap blocks were freed -- no leaks are possible
==1311==
==1311== For lists of detected and suppressed errors, rerun with: -s
==1311== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Practical-08$ |
```

## Cow.c codes

```
GNU nano 8.7.1                                COW.C
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int *data = (int *)malloc(sizeof(int));

    *data = 100;

    printf("Before fork:\n");
    printf("Parent: value = %d, address = %p\n",
        *data, (void *)data);

    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed.\n");
        free(data);
        return 1;
    }

    if (pid == 0) {
        printf("\nChild before modification:\n");
        printf("Child: value = %d, address = %p\n",
            *data, (void *)data);

        *data = 200;

        printf("Child after modification:\n");
        printf("Child: value = %d, address = %p\n",
            *data, (void *)data);

        free(data);
    }
    else {
        wait(NULL);

        printf("\nParent after child modification:\n");
        printf("Parent: value = %d, address = %p\n",
```

```
        printf("\nParent after child modification:\n");
        printf("Parent: value = %d, address = %p\n",
            *data, (void *)data);

        free(data);
    }

    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

Output:

```
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Practical-08$ nano cow.c
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Practical-08$ gcc cow.c -o cow
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Practical-08$ ./cow
Before fork:
Parent: value = 100, address = 0x63b045e71010

Child before modification:
Child: value = 100, address = 0x63b045e71010
Child after modification:
Child: value = 200, address = 0x63b045e71010

Parent after child modification:
Parent: value = 100, address = 0x63b045e71010
```